

Master Project Specification Document (MPSD)

Project: ISF Playground - Combined Sprint 2 & Sprint 5 **Version:** 3.0 (Unified) **Date:** September 16, 2025 **Sprint Duration:** 30 Days (Parallel Execution)

1. MPSD Introduction & Combined Sprint Overview

1.1. Purpose of This Master Project Specification Document

This Master Project Specification Document (MPSD) serves as the exhaustive, unambiguous, and universally agreed-upon blueprint for the parallel execution of ISF Playground Sprint 2 and Sprint 5. It is the single source of truth, meticulously detailing every facet of the features planned for this combined 30-day development effort. Its primary purpose is to ensure complete alignment among all stakeholders—including the Client, Project Manager, Design Team, and Development Team—before the parallel sprint development work commences. This MPSD will be the definitive reference for scope management, enabling the Project Manager to clearly delineate agreed-upon features from out-of-scope requests, thereby ensuring focused and efficient development across both sprint deliverables simultaneously.

1.2. Project Overview: ISF Playground - Combined Sprint 2 & 5

This combined sprint represents a strategic acceleration of the ISF Playground development roadmap, executing Sprint 2 and Sprint 5 features in parallel over a 30-day period. Sprint 2 focuses on the comprehensive Learning Management System (LMS) and enhanced Amma role capabilities, while Sprint 5 completes the platform's virtual economy loop with the ISF Shop module. This parallel execution approach leverages the modular architecture established in Sprint 1 and the completed WTF module, allowing independent development streams while maintaining system cohesion. The entire system continues to be built as an Electron.js desktop application with a Node.js backend and MongoDB database, prioritizing offline capabilities for core student interactions.

1.3. Combined Sprint Goals & Objectives

The primary goals for this unified 30-day sprint are:

Sprint 2 Core Objectives:

- 1. Deliver a Functional LMS Core:** Implement robust course management (creation, assignment, content population), student enrollment, and assessment functionalities, including flexible course assignment for individual weaker students to create personalized learning paths.
- 2. Enable Rich Student Learning Experiences:** Provide students with an interactive and engaging platform to access diverse course types (Computer Apps, Art, Spoken English, Life Skills) primarily through voice and intuitive

UI, including offline access to core learning activities. 3. **Integrate ISF Coin Rewards:** Implement the system for students to earn digital ISF Coins through LMS activity completion and coach grading. 4. **Enhance Amma Role Capabilities:** Introduce individual Amma accounts, improve query handling with better categorization and SLA management, and integrate voice note communication.

Sprint 5 Core Objectives: 1. **Launch a Functional Virtual Economy:** Implement the end-to-end ISF Shop module, allowing students to spend their earned ISF Coins on physical or digital rewards. 2. **Empower Admins with E-Commerce Tools:** Provide Administrators with robust interfaces to manage shop products, control inventory levels, and track the flow of the virtual currency. 3. **Provide Insightful Economic Reports:** Develop a “Coin Distribution Reports” module to give Admins a clear view of student engagement, coin circulation, and reward redemption patterns. 4. **Complete System Integration:** Finalize all system-wide features including enhanced reporting, security audits, and performance optimization for production release.

1.4. Parallel Development Strategy

To achieve the ambitious 30-day timeline, the development will be organized into two parallel workstreams:

Workstream A - LMS & Learning Features (Sprint 2): - Week 1-2: Core LMS infrastructure, course management, content management - Week 2-3: Student learning interfaces for all four course types - Week 3-4: Grading system, reporting, Amma enhancements - Week 4: Integration testing and refinement

Workstream B - Commerce & Economy Features (Sprint 5): - Week 1-2: ISF Shop frontend, product catalog, shopping cart - Week 2-3: Admin product management, inventory system - Week 3-4: Coin economy reports, transaction management - Week 4: Integration with LMS coin earning, end-to-end testing

Cross-Stream Dependencies: - Shared ISF Coin wallet system - Unified notification center - Common user authentication and session management - Integrated reporting dashboard

2. Target Users/Personas for Combined Sprint Features

Primary Personas:

- **Student:** The primary beneficiary of both sprints. Interacts with LMS courses to learn and earn ISF Coins (Sprint 2), then uses the ISF Shop to redeem earned coins for rewards (Sprint 5). Experiences a complete learning-to-reward cycle within the 30-day implementation.
- **Administrator (Admin):** The system orchestrator with expanded responsibilities across both sprints. Manages all course creation and content

(Sprint 2), oversees the ISF Shop inventory and products (Sprint 5), and has access to comprehensive reporting across both learning and commerce domains.

- **Coach:** Focused primarily on Sprint 2 features. Assigns courses to Balagruhas or specific students, grades subjective student work via the “Syllabus Tracker,” monitors student progress, and manually awards ISF Coins. Has visibility into student shop purchases for counseling purposes.

Secondary Personas:

- **Amma:** Enhanced through Sprint 2 with individual accounts, improved query handling, SLA-based task management, and voice note communication. Can view student coin spending patterns to understand welfare needs.
 - **Playground Manager (PM):** System health monitor introduced in Sprint 2. All application errors, stakeholder feedback, and suggestions from both LMS and Shop modules are automatically logged and assigned as tasks for resolution.
-

3. High-Level Combined Sprint Scope

3.1. What’s In Scope for the 30-Day Combined Sprint

Sprint 2 - LMS & Communication Features: **LMS - Student Experience:** * Student Homepage & Main Course Category Navigation (built to client UI specifications) * Full interaction with “Computer Apps,” “Art,” “Spoken English,” and “Life Skills” courses * Voice-enabled multiple-choice questions with audio buttons * Digital ISF Coin accumulation in student wallet * Offline-first architecture for core learning activities

LMS - Admin & Coach Functionalities: * **Course Management (Admin Only):** Full CRUD operations for courses with module/chapter structure * **Course Assignment (Admin & Coach):** Flexible assignment to Balagruhas and individual students * **Content Management Module (Admin Only):** Upload/manage Video, PDF, Document, Image, Audio, Text, Links * **Quiz System (Admin Only):** Create assessments with various question types and passing criteria * **Coach Grading (Coach Only):** Grade subjective submissions via “Syllabus Tracker” with manual coin awards * **Course Reporting System (Admin & Coach):** Performance dashboards with PDF/CSV export

Amma & Communication Features: * Individual Amma Accounts with self-registration and Admin approval workflow * Enhanced Query Handling with reclassification and multi-tagging capabilities * SLA-based Task Management with automatic reassignment * Voice Communication for all roles (press-and-

hold recording) * Admin Broadcast (“Mann ki Baat”) for system-wide announcements * Automated WhatsApp notifications for daily schedules

System-Wide (Sprint 2): * In-App Notification Center with dual-display system * Translation Module for course content (Hindi, English, Marathi) * Playground Manager role for error tracking * Child-friendly error messages

Sprint 5 - E-Commerce & Economy Features: ISF Shop Module (Student Experience): * Shop access via main menu with product catalog grid view * Advanced filtering and sorting (by category, price) * Multi-item shopping cart with quantity management * Secure checkout using ISF Coins with balance validation * “My Purchases” section with order history and digital receipts * Wishlist functionality for saving desired items

ISF Shop & Inventory Management (Admin Experience): * Complete product CRUD operations with image management * Real-time inventory tracking with stock level monitoring * Automatic stock decrementation on purchase * Out-of-stock handling and display * Bulk product import/export capabilities * Product categorization and tagging system

Coin Distribution Reports: * Comprehensive coin economy dashboard * Total earned vs. spent metrics with circulation tracking * Student leaderboards for earners and spenders * Top redeemed products analytics * Individual student transaction history with detailed logs * Exportable reports in PDF and CSV formats

Final System Refinements (Sprint 5): * Security audit across all new features * Performance optimization for target hardware * Complete technical documentation * API documentation with Swagger/OpenAPI * User training materials preparation

3.2. What’s Out of Scope for the Combined Sprint

Sprint 2 Exclusions: * Continuous “Always-On” Facial Recognition during assessments * System-Wide Archiving Framework (limited to course archiving only) * Full UI Internationalization (limited to content translation) * Life Skills “Learn then Test” expanded model * Task “Revision Mode” for half coins * AI-Powered Transcription of voice notes

Sprint 5 Exclusions: * Physical Item Fulfillment & Logistics * Real-money Payment Gateway Integration * Supplier/Vendor Management System * Dropshipping or Third-party Marketplace Integration * Product Reviews and Ratings System * Promotional Codes or Discount System

4. Target Audience for this MPSD

- **Primary:** Project Manager (for sprint coordination, resource allocation, client communication, and integrated scope management across both workstreams)
 - **Secondary:**
 - **Development Team (Frontend & Backend):** For precise implementation details across both feature sets
 - **QA Team:** For comprehensive test planning covering LMS-to-Shop integration scenarios
 - **Design Team:** For ensuring UI/UX consistency across learning and commerce experiences
 - **Client (ISF):** For final approval and understanding of the accelerated delivery timeline
-

5. Document Conventions

- **UI Elements:** Referred to by their visible label text in “quotation marks” or by descriptive name with suggested IDs (e.g., btn-create-course, btn-add-to-cart)
 - **User Roles:** Capitalized consistently (Student, Coach, Admin, Amma, Playground Manager)
 - **Placeholders:** Dynamic data shown in [square_brackets] (e.g., [Course-Name], [ProductName], [CoinBalance])
 - **API Endpoints:** Represented as METHOD /path/to/endpoint with versioning where applicable
 - **Sprint Attribution:** Features are tagged with [S2] for Sprint 2 or [S5] for Sprint 5 to maintain clarity
 - **Cross-Sprint Dependencies:** Marked with [SHARED] when a feature impacts both sprints
-

6. References to Source Documents

This combined MPSD synthesizes and integrates information from:

- Sprint 2 MPSD v2.0 (August 19, 2025)
- Sprint 5 MPSD v1.0 (August 19, 2025)
- Client-provided UI mockups (definitive source for all UI implementation)
- ‘Playground Platform - 5 Sprint Plan - 16 February’
- ‘WTF (Wall for Thrust towards Fame) with client comments’
- ‘ISF Sprint 2 - Overview and Scope’
- All LMS feature breakdown documents
- Completed Sprint 1 implementation and codebase

7. Global Elements & Standards (Combined Sprint Context)

7.1. Branding Guidelines

- **Consistency Requirement:** Both LMS and Shop modules must maintain identical branding
- **Color Palette:** Primary - Green, Secondary - Blue, Accent - Gray (applied uniformly)
- **Typography:** Consistent font families across learning and commerce interfaces
- **Logo Placement:** ISF Playground logo consistent in all module headers

7.2. Responsive Design & Performance

- **Target Resolution:** 1366x768 (primary target for all new features)
- **Hardware Benchmark:** Core i3 4th Gen, 8GB DDR3 RAM, 256GB SSD
- **Performance Requirements:**
 - Page load times < 3 seconds for both LMS and Shop
 - Cart operations < 500ms response time
 - Course content loading with progressive enhancement

7.3. Accessibility Standards

- **WCAG 2.1 Level AA:** Applied consistently across both sprint deliverables
- **Keyboard Navigation:** Full support in course navigation and shop browsing
- **Screen Reader Compatibility:** All interactive elements properly labeled
- **Color Contrast:** Minimum 4.5:1 for normal text, 3:1 for large text

7.4. Unified Navigation Structure

- **Student Homepage Enhancement:**
 - [S2] Persistent header with emotion emojis, voice chat, homework buttons
 - [S5] New “ISF Shop” button integrated into main navigation
 - [SHARED] ISF Coin balance display updated real-time across both modules
 - [SHARED] Notification bell showing alerts from both LMS and Shop
- **Admin Dashboard Unification:**
 - [S2] Course Management, Content Management, Broadcast Message
 - [S5] Shop Management, Inventory Control, Product Analytics

- [SHARED] Unified Reports section combining learning and commerce metrics
-

Detailed Feature & Module Breakdown

Section A: Sprint 2 - LMS & Communication Features

8. LMS - Student Homepage & Course Navigation [S2]

- **Feature ID:** S2-LMS-STU-001
- **Feature Name:** Student Homepage & Main Course Category Selection
- **Development Timeline:** Week 1 (Days 1-5)

User Story 1: “As a Student, after logging in via facial recognition, I want to see a clear and simple homepage that shows my learning options and my current coin balance, so I can understand my status and available activities.”

Acceptance Criteria: * **AC1.1:** Upon successful facial recognition, the student is directed to the enhanced LMS homepage within 2 seconds * **AC1.2:** The homepage UI strictly matches the client-provided design mockups * **AC1.3:** The persistent Title Bar displays: * Student Name and Profile Picture * Active session timer (counting up) * ISF Coin Balance (dynamically updated) * Notification bell icon with count badge * Window controls (minimize, maximize, close) * **AC1.4:** The Toolbar (below Title Bar) contains: * Five emotion emoji buttons (100x100px each, 10px gap) * “Voice Chat” button with pending message badge * “Homework” button * [S5] “ISF Shop” button (added for Sprint 5 integration) * **AC1.5:** System checks for incomplete tasks and auto-resumes if found

User Story 2: “As a Student, I want to click on course categories to view activities, with the system preventing accidental navigation away from active tasks.”

Acceptance Criteria: * **AC2.1:** Four main course category buttons are displayed: * Computer Apps (with computer icon) * Art (with palette icon) * Spoken English (with microphone icon) * Life Skills (with heart icon) * **AC2.2:** Each button shows a visual indicator if it contains active/new content * **AC2.3:** Clicking a category triggers SPA-style content replacement (no page reload) * **AC2.4:** If student is mid-task and clicks another category, confirmation modal appears: * Message: “Are you sure you want to switch? Your current progress will be saved.” * Buttons: “Yes, switch” | “No, stay”

Technical Implementation Details: Frontend Components:

// Component Structure
– StudentHomepage/

- TitleBar/
 - StudentProfile
 - SessionTimer
 - CoinBalance (shared with S5)
 - NotificationBell
- Toolbar/
 - EmotionSelector
 - VoiceChat
 - Homework
 - ShopButton (S5)
- CourseGrid/
 - CategoryCard

API Endpoints: * GET /api/lms/student/dashboard - Fetch homepage data
 * GET /api/lms/student/course-categories - Get available categories
 * GET /api/lms/student/incomplete-tasks - Check for resumable tasks
 * POST /api/lms/student/session/switch - Save state when switching

Data Model Updates:

```
{
  studentDashboard: {
    studentId: ObjectId,
    currentSession: {
      startTime: Date,
      lastActivity: Date,
      activeTask: ObjectId
    },
    coinBalance: Number, // Shared with S5
    courseProgress: {
      computerApps: { completed: Number, total: Number },
      art: { completed: Number, total: Number },
      spokenEnglish: { completed: Number, total: Number },
      lifeSkills: { completed: Number, total: Number }
    }
  }
}
```

9. LMS - Computer Apps Courses (Student Interaction) [S2]

- **Feature ID:** S2-LMS-STU-002
- **Feature Name:** Computer Apps Course Interaction (Student View)
- **Development Timeline:** Week 1-2 (Days 3-8)

User Story 1: “As a Student, I want to navigate through Computer Apps with a three-pane layout showing apps, levels, and tasks, with clear progress indicators.”

Acceptance Criteria:

- * **AC1.1:** Three-pane layout implementation:
 - * Left Column (200px): Application list
 - * Top Row (80px): Level selection buttons
 - * Main Palette: Task grid with progress indicators
- * **AC1.2:** Initial state shows blinking prompt: “Please select an app”
- * **AC1.3:** Visual feedback on selection:
 - * Selected app highlights with primary color
 - * In-progress level shows subtle pulse animation (CSS keyframes)
 - * Completed tasks show green checkmark overlay

User Story 2: “As a Student, I want to see my performance metrics including ranking compared to other students.”

Acceptance Criteria:

- * **AC2.1:** Each completed task displays performance bar showing:
 - * Time Taken (format: “MM:SS”)
 - * ISF Coins Earned (with coin icon)
 - * Rank (format: “Your Rank: X out of Y students”)
- * **AC2.2:** Progress lines under each task:
 - * Completed: Full green line (100% width)
 - * In-Progress: Half green, half gray (50% green gradient)
 - * Not Started: Red line (100% width)

Technical Implementation: Frontend State Management:

```
const ComputerAppsState = {
  selectedApp: null,
  selectedLevel: null,
  tasks: [],
  studentProgress: {
    taskId: {
      status: 'completed|in-progress|not-started',
      timeSpent: seconds,
      coinsEarned: number,
      rank: { position: number, total: number },
      lastAttempt: Date
    }
  }
};
```

Real-time Rank Calculation:

```
// Backend aggregation for ranking
db.taskProgress.aggregate([
  { $match: { taskId: taskId } },
  { $sort: { timeSpent: 1 } },
  { $group: {
    _id: "$taskId",
    rankings: { $push: { studentId: "$studentId", time: "$timeSpent" } }
  }}
]);
```

10. LMS - Art Courses with Artweaver Integration [S2]

- **Feature ID:** S2-LMS-STU-003
- **Feature Name:** Art Course Interaction & Artweaver Integration
- **Development Timeline:** Week 1-2 (Days 5-10)

User Story 1: “As a Student, I want to use a graphics pad with Artweaver to create digital art that syncs with the ISF Playground in real-time.”

Acceptance Criteria: * **AC1.1:** Four art modes available: * Workshops (structured lessons) * Free Sketch (creative time) * Art Stories (narrative art) * Competition (timed challenges) * **AC1.2:** Split-view layout: * Left (40%): Instruction palette with reference image/prompt * Right (60%): Real-time canvas mirror from Artweaver * **AC1.3:** Graphics pad detection and configuration: * Auto-detect USB graphics pad on task launch * Fallback mouse input if pad unavailable * Pressure sensitivity support (if hardware capable)

Technical Integration Architecture: Artweaver Bridge Implementation:

```
// Electron main process
const ArtWeaverBridge = {
  launchArtweaver: async () => {
    const artWeaverPath = process.platform === 'win32'
      ? 'C:\\Program Files\\Artweaver\\Artweaver.exe'
      : null;

    // Launch with specific canvas size
    spawn(artWeaverPath, ['--canvas-size=800x600']);
  },

  captureCanvas: async () => {
    // Screenshot Artweaver window
    const screenshot = await captureWindow('Artweaver');
    return screenshot.toPNG();
  },

  syncInterval: setInterval(() => {
    // Sync every 500ms for near real-time
    this.captureCanvas().then(data => {
      mainWindow.webContents.send('canvas-update', data);
    });
  }, 500)
};
```

11. LMS - Spoken English with Video Recording [S2]

- **Feature ID:** S2-LMS-STU-004
- **Feature Name:** Spoken English Course Video Performance
- **Development Timeline:** Week 2 (Days 8-12)

User Story 1: “As a Student, I want to record video performances with preview and re-record options before submission.”

Acceptance Criteria: * **AC1.1:** Video recording interface includes: * Live camera preview (640x480 minimum) * Recording indicator (red dot + timer) * Audio level meter * **AC1.2:** Recording workflow: * “Start Recording” begins capture * Auto-stop at 3 minutes maximum * “Stop Recording” ends capture * **AC1.3:** Review options post-recording: * “Play My Video” for preview * “Re-record” discards and restarts * “Submit Video” saves and grades

WebRTC Implementation:

```
const VideoRecorder = {
  constraints: {
    video: { width: 640, height: 480, facingMode: "user" },
    audio: { echoCancellation: true, noiseSuppression: true }
  },

  startRecording: async function() {
    const stream = await navigator.mediaDevices.getUserMedia(this.constraints);
    this.mediaRecorder = new MediaRecorder(stream, {
      mimeType: 'video/webm;codecs=vp8,opus'
    });

    this.mediaRecorder.ondataavailable = (event) => {
      this.chunks.push(event.data);
    };

    this.mediaRecorder.start();
    this.startTimer();
  }
};
```

12. LMS - Life Skills Interactive Learning [S2]

- **Feature ID:** S2-LMS-STU-005
- **Feature Name:** Life Skills Course with Voice Responses
- **Development Timeline:** Week 2 (Days 10-14)

User Story 1: “As a Student, I want to answer Life Skills questions using voice notes like WhatsApp.”

Acceptance Criteria: * **AC1.1:** Voice recording using press-and-hold: * Mouse down: Start recording * Mouse up: Stop recording * Visual feedback during recording (pulsing mic icon) * **AC1.2:** Recording constraints: * Maximum 1 minute duration * Auto-stop at limit with notification * Minimum 3 seconds for valid response * **AC1.3:** Mandatory playback before submission: * “Submit” button disabled until played * Option to re-record unlimited times

13. Course Management System (Admin) [S2]

- **Feature ID:** S2-LMS-AC-001
- **Feature Name:** Complete Course Builder and Management
- **Development Timeline:** Week 1-2 (Days 1-10)

User Story 1 (Admin): “As an Admin, I need a comprehensive course builder with drag-and-drop module/chapter organization.”

Acceptance Criteria: * **AC1.1:** Course Builder Interface includes: * Course metadata form (title, description, category, thumbnail) * Hierarchical structure: Course > Modules > Chapters > Content Items * Drag-and-drop reordering at all levels * Real-time preview of course structure * **AC1.2:** Content type support per chapter: * Video (MP4, MOV, max 500MB) * Documents (PDF, DOC, PPT, max 50MB) * Audio (MP3, WAV, max 100MB) * Images (JPG, PNG, max 10MB) * External links (YouTube, educational sites) * Quizzes (integrated with quiz builder) * **AC1.3:** Publishing workflow: * Save as Draft (auto-save every 30 seconds) * Preview mode for testing * Publish with scheduling option * Archive with data retention

Course Data Model:

```
const CourseSchema = {
  _id: ObjectId,
  title: { en: String, hi: String, mr: String }, // Multilingual
  description: { en: String, hi: String, mr: String },
  category: String, // 'computer-apps', 'art', 'spoken-english', 'life-skills'
  thumbnail: String, // S3 URL
  status: String, // 'draft', 'published', 'archived'

  modules: [{
    moduleId: String,
    title: { en: String, hi: String, mr: String },
    order: Number,
    chapters: [{
      chapterId: String,
      title: { en: String, hi: String, mr: String },
      order: Number,
      contentItems: [{
        itemId: String,
```

```

        type: String, // 'video', 'document', 'quiz', etc.
        title: String,
        url: String, // S3 or external
        duration: Number, // estimated minutes
        coinReward: {
            enabled: Boolean,
            amount: Number,
            condition: String // 'completion', 'passing-grade'
        }
    }
}
}],

assignments: [{
    balagruhaId: ObjectId,
    studentIds: [ObjectId], // specific students or empty for all
    startDate: Date,
    dueDate: Date
}],

metrics: {
    totalEnrolled: Number,
    avgCompletion: Number,
    avgScore: Number,
    totalCoinsAwarded: Number
},

createdBy: ObjectId,
createdAt: Date,
updatedAt: Date
};

```

14. Coach Grading System via Syllabus Tracker [S2]

- **Feature ID:** S2-LMS-COACH-002
- **Feature Name:** Subjective Task Grading Interface
- **Development Timeline:** Week 2-3 (Days 12-18)

User Story 1 (Coach): “As a Coach, I need the Syllabus Tracker to show pending submissions with smart prioritization and bulk grading capabilities.”

Acceptance Criteria: * **AC1.1:** Syllabus Tracker dashboard shows: * Pending submissions count per Balagruha (badge on button) * Priority sorting (oldest first, SLA warnings) * Filter by: Subject, Student, Submission type * Bulk actions: Grade multiple with same score * **AC1.2:** Individual grading interface: * Full-screen view of submitted work * Side panel for grading controls * Grade

scale: A+ to F (mapped to percentages) * Manual ISF Coin award (0-100 coins)
* Optional feedback text (max 500 chars) * Voice note feedback option (max 1 minute) * **AC1.3:** Grading workflow optimization: * Keyboard shortcuts (1-5 for grades, Enter to submit) * Auto-advance to next submission * Save draft grades for later

15. Translation Module [S2]

- **Feature ID:** S2-LMS-SYS-002
- **Feature Name:** Multi-language Content Support
- **Development Timeline:** Week 3 (Days 15-18)

Implementation Strategy:

```
const TranslationManager = {
  supportedLanguages: ['en', 'hi', 'mr'],

  translateContent: async (content, targetLang) => {
    // Check cache first
    if (cache[content.id]?.[targetLang]) {
      return cache[content.id][targetLang];
    }

    // Manual translation by Admin
    return content[targetLang] || content['en']; // Fallback to English
  },

  studentPreference: async (studentId) => {
    const student = await db.students.findById(studentId);
    return student.preferredLanguage || 'en';
  }
};
```

Section B: Sprint 5 - ISF Shop & Economy Features

16. ISF Shop Storefront (Student) [S5]

- **Feature ID:** S5-SHOP-STU-001
- **Feature Name:** Student Shopping Experience
- **Development Timeline:** Week 1-2 (Days 1-10)

User Story 1: “As a Student, I want an Amazon-like shopping experience where I can browse, filter, and purchase items with my ISF Coins.”

Acceptance Criteria: * **AC1.1:** Shop landing page features: * Grid layout (3 columns on 1366x768) * Product cards with image, name, price * “Out

of Stock” overlay when quantity = 0 * Hover effects showing quick details * **AC1.2:** Advanced filtering and sorting: * Category filters (Toys, Books, Stationery, Sports, Digital) * Price range slider (0-500 coins) * Sort options: Price (Low/High), Newest, Most Popular * Active filter pills with remove option * **AC1.3:** Product detail page: * Image gallery with zoom capability * Detailed description with specifications * Stock availability indicator * Related products section * Add to Cart with quantity selector * Add to Wishlist toggle

User Story 2: “As a Student, I want a persistent shopping cart that saves my selections even if I leave the shop.”

Acceptance Criteria: * **AC2.1:** Shopping cart features: * Persistent storage in local Memory Layer * Cart icon with item count badge * Sliding cart drawer (right side) * Quick cart preview on hover * **AC2.2:** Cart management: * Quantity adjustment (+/- buttons) * Remove item with confirmation * Save for later option * Estimated coins display * Stock validation on cart open

User Story 3: “As a Student, I want a secure checkout process that clearly shows my coin balance and purchase confirmation.”

Acceptance Criteria: * **AC3.1:** Checkout flow (3 steps): * Step 1: Review cart items * Step 2: Confirm coin payment * Step 3: Success confirmation * **AC3.2:** Payment validation: * Current balance: [X] coins (green if sufficient, red if not) * Order total: [Y] coins * Remaining after purchase: [Z] coins * Insufficient funds handling with suggestion to earn more * **AC3.3:** Order confirmation: * Order ID generation (format: ORD-YYYYMMDD-XXXXXX) * Email-style receipt display * Print receipt option (PDF generation) * Continue shopping / View orders buttons

Shopping Cart State Management:

```
const ShoppingCartState = {
  cartId: String,
  studentId: ObjectId,
  items: [{
    productId: ObjectId,
    productName: String,
    quantity: Number,
    pricePerUnit: Number,
    thumbnail: String,
    stockAvailable: Number
  }],
  totalItems: Number,
  totalCost: Number,
  lastModified: Date,
  savedForLater: [],
```

```

// Methods
addItem: function(product, quantity) {
  // Check stock
  // Add or update quantity
  // Recalculate totals
  // Sync to backend
},

validateStock: async function() {
  // Real-time stock check
  // Remove/adjust unavailable items
  // Notify user of changes
}
};

```

17. ISF Shop Product Management (Admin) [S5]

- **Feature ID:** S5-SHOP-ADM-001
- **Feature Name:** E-Commerce Administration
- **Development Timeline:** Week 1-2 (Days 3-12)

User Story 1 (Admin): “As an Admin, I need a comprehensive product management system similar to standard e-commerce platforms.”

Acceptance Criteria: * **AC1.1:** Product creation form: * Basic Info: Name, SKU, Category, Brand * Pricing: Cost in ISF Coins * Images: Multiple upload (primary + gallery) * Description: Rich text editor with formatting * Specifications: Key-value pairs * SEO: Meta title, description (for internal search) * **AC1.2:** Product variations support: * Size options (S, M, L, XL) * Color variants with separate images * Individual stock tracking per variant * **AC1.3:** Bulk operations: * CSV import/export for products * Bulk price updates * Bulk stock adjustments * Bulk category changes

Product Data Model:

```

const ProductSchema = {
  _id: ObjectId,
  sku: String, // Unique identifier
  name: String,
  category: String,
  subcategory: String,
  brand: String,

  pricing: {
    basePrice: Number, // in ISF Coins
    discountedPrice: Number,
    discountPercentage: Number
  }
}

```

```

    },

    media: {
      primaryImage: String, // S3 URL
      gallery: [String], // Additional images
      videos: [String] // Product demos
    },

    description: {
      short: String, // For cards
      long: String, // Rich text for detail page
      highlights: [String], // Bullet points
      specifications: Map // Key-value pairs
    },

    inventory: {
      trackInventory: Boolean,
      stockQuantity: Number,
      lowStockThreshold: Number,
      allowBackorder: Boolean,
      variants: [{
        variantId: String,
        type: String, // 'size', 'color'
        value: String,
        stock: Number,
        additionalPrice: Number
      }]
    },

    metadata: {
      isActive: Boolean,
      isFeatured: Boolean,
      tags: [String],
      createdAt: Date,
      updatedAt: Date,
      createdBy: ObjectId
    },

    analytics: {
      views: Number,
      purchases: Number,
      wishlistAdds: Number,
      rating: Number,
      lastPurchased: Date
    }
  };

```

18. Inventory Management System [S5]

- **Feature ID:** S5-INV-ADM-001
- **Feature Name:** Real-time Inventory Control
- **Development Timeline:** Week 2 (Days 10-14)

User Story 1 (System): “As a System, I must maintain accurate inventory levels with automatic updates and low-stock alerts.”

Acceptance Criteria: * **AC1.1:** Automatic stock management: * Decrement on successful purchase * Restore on order cancellation (if implemented) * Reserve stock during checkout (5-minute hold) * Release reserved stock on timeout * **AC1.2:** Low stock alerts: * Threshold configuration per product * Dashboard notification when breached * Email alert to designated admin * Auto-generate purchase recommendation * **AC1.3:** Stock audit trail: * Log all stock changes with timestamp * Track reason (purchase, manual adjustment, return) * User who made change * Previous and new values

Inventory Transaction Model:

```
const InventoryTransaction = {
  _id: ObjectId,
  productId: ObjectId,
  variantId: String,
  transactionType: String, // 'purchase', 'adjustment', 'return', 'reserved'
  quantity: Number, // positive for additions, negative for deductions
  previousStock: Number,
  newStock: Number,
  reference: {
    type: String, // 'order', 'manual', 'system'
    id: ObjectId // orderId or userId
  },
  notes: String,
  performedBy: ObjectId,
  timestamp: Date
};
```

19. Coin Distribution Reports [S5]

- **Feature ID:** S5-REP-ADM-001
- **Feature Name:** Virtual Economy Analytics
- **Development Timeline:** Week 3 (Days 15-20)

User Story 1 (Admin): “As an Admin, I need comprehensive insights into the coin economy to ensure balanced reward distribution.”

Acceptance Criteria: * **AC1.1:** Economy Dashboard displays: * Total Coins in Circulation (real-time) * Daily/Weekly/Monthly earning trends *

Daily/Weekly/Monthly spending trends * Coins-to-Activity ratio (health metric) * Average student wallet balance * **AC1.2:** Student Analytics: * Top 10 Earners leaderboard with earning sources * Top 10 Spenders with purchase categories * Inactive wallets (no activity > 30 days) * Student journey map (earn → save → spend cycle) * **AC1.3:** Product Performance: * Best selling products (by units and revenue) * Product conversion rates (views to purchases) * Category performance comparison * Stock turnover rate * **AC1.4:** Detailed Transaction Logs: * Searchable by student, date range, type * Export capabilities (PDF, CSV) * Transaction details drill-down * Audit trail for coin adjustments

Analytics Aggregation Queries:

```
// Daily coin flow aggregation
db.coinTransactions.aggregate([
  { $match: {
    timestamp: {
      $gte: startOfDay,
      $lt: endOfDay
    }
  }},
  { $group: {
    _id: {
      type: "$transactionType",
      hour: { $hour: "$timestamp" }
    },
    total: { $sum: "$amount" },
    count: { $sum: 1 }
  }},
  { $project: {
    hour: "$_id.hour",
    earned: { $cond: [{ $eq: ["$_id.type", "earned"] }, "$total", 0] },
    spent: { $cond: [{ $eq: ["$_id.type", "spent"] }, "$total", 0] }
  }}
]);
```

Section C: Shared & System-Wide Features

20. Unified ISF Coin Wallet System [SHARED]

- **Feature ID:** S2-S5-WALLET-001
- **Feature Name:** Integrated Coin Management
- **Development Timeline:** Week 1 (Days 1-5) - Critical path

Implementation Requirements:

- Real-time balance synchronization between LMS earning and Shop spending
- Transaction atomicity to prevent double-spending
- Offline transaction queuing with reconciliation
- Audit log for all coin movements

Wallet State Management:

```
const WalletService = {
  // Atomic transaction for coin operations
  transferCoins: async (studentId, amount, type, reference) => {
    const session = await mongoose.startSession();
    session.startTransaction();

    try {
      // Get current balance with lock
      const wallet = await Wallet.findOne({ studentId }).session(session);

      // Validate sufficient funds for spending
      if (type === 'spend' && wallet.balance < amount) {
        throw new Error('Insufficient funds');
      }

      // Update balance
      wallet.balance += (type === 'earn' ? amount : -amount);
      await wallet.save({ session });

      // Log transaction
      await TransactionLog.create([
        {
          studentId,
          type,
          amount,
          reference,
          balanceAfter: wallet.balance,
          timestamp: new Date()
        }
      ], { session });

      await session.commitTransaction();

      // Emit real-time update
      io.to(`student-${studentId}`).emit('wallet-update', wallet.balance);

      return wallet.balance;
    } catch (error) {
      await session.abortTransaction();
      throw error;
    }
  }
}
```

```

    } finally {
        session.endSession();
    }
}
};

```

21. Enhanced Notification Center [SHARED]

- **Feature ID:** S2-S5-NOTIF-001
- **Feature Name:** Unified Notification System
- **Development Timeline:** Week 2 (Days 8-12)

Notification Types & Templates: Sprint 2 Notifications:

```

const LMSNotifications = {
  courseAssigned: {
    title: "New Course Available!",
    body: "You've been assigned '[CourseName]'",
    icon: "book",
    action: "/lms/courses/[courseId]"
  },

  gradeReceived: {
    title: "Work Graded!",
    body: "Your [TaskType] received [Grade] and [Coins] coins!",
    icon: "star",
    action: "/lms/grades"
  },

  voiceNoteReceived: {
    title: "New Voice Message",
    body: "You have a voice note from [SenderName]",
    icon: "mic",
    action: "/communication/voice/[noteId]"
  }
};

```

Sprint 5 Notifications:

```

const ShopNotifications = {
  orderConfirmed: {
    title: "Order Successful!",
    body: "Order #[OrderId] confirmed. [Items] items purchased.",
    icon: "shopping-bag",
    action: "/shop/orders/[orderId]"
  },
};

```

```

lowBalance: {
  title: "Low Coin Balance",
  body: "You need [Amount] more coins for your cart items",
  icon: "alert",
  action: "/lms/earn-more"
},

backInStock: {
  title: "Item Available!",
  body: "[ProductName] from your wishlist is back in stock",
  icon: "heart",
  action: "/shop/product/[productId]"
}
};

```

22. Voice Communication System [S2]

- **Feature ID:** S2-SYS-VC-001
- **Feature Name:** WhatsApp-Style Voice Notes
- **Development Timeline:** Week 2-3 (Days 10-16)

Technical Implementation:

```

const VoiceNoteService = {
  recordingConstraints: {
    audio: {
      echoCancellation: true,
      noiseSuppression: true,
      sampleRate: 44100,
      channelCount: 1
    }
  },

  startRecording: function() {
    navigator.mediaDevices.getUserMedia(this.recordingConstraints)
      .then(stream => {
        this.mediaRecorder = new MediaRecorder(stream, {
          mimeType: 'audio/webm;codecs=opus'
        });

        this.mediaRecorder.start();
        this.visualizeAudio(stream); // Waveform visualization
      });
  },

  processRecording: async function(audioBlob) {

```

```

    // Compress audio
    const compressed = await this.compressAudio(audioBlob);

    // Upload to S3
    const url = await this.uploadToS3(compressed);

    // Send notification
    await NotificationService.send({
      type: 'voice-note',
      recipientId: this.recipientId,
      audioUrl: url,
      duration: this.duration
    });
  }
};

```

23. Automated WhatsApp Integration [S2]

- **Feature ID:** S2-SYS-WAPP-001
- **Feature Name:** Schedule Distribution via WhatsApp
- **Development Timeline:** Week 3 (Days 18-21)

Integration Architecture:

```

const WhatsAppService = {
  provider: 'twilio', // or '360dialog'

  sendScheduleNotification: async (balagruhaId, schedule) => {
    const balagruha = await Balagruha.findById(balagruhaId);

    if (!balagruha.whatsappGroupId) {
      logger.warn(`No WhatsApp group for Balagruha ${balagruhaId}`);
      return;
    }

    const message = formatScheduleMessage(schedule);

    try {
      await twilioClient.messages.create({
        from: 'whatsapp:+14155238886', // Twilio sandbox number
        to: `whatsapp:${balagruha.whatsappGroupId}`,
        body: message,
        mediaUrl: schedule.imageUrl // Optional schedule image
      });

      logger.info(`Schedule sent to Balagruha ${balagruhaId}`);
    }
  }
};

```

```

    } catch (error) {
      logger.error(`WhatsApp send failed: ${error.message}`);
      // Queue for retry
      await RetryQueue.add({ type: 'whatsapp', payload: { balagruhaId, schedule } });
    }
  }
};

```

24. Error Handling & PM Task System [S2]

- **Feature ID:** S2-SYS-PM-001
- **Feature Name:** Centralized Error Management
- **Development Timeline:** Week 1 (Days 3-5)

Global Error Handler Implementation:

```

// Frontend Error Boundary
class GlobalErrorBoundary extends React.Component {
  componentDidCatch(error, errorInfo) {
    // Log to backend
    API.post('/api/system/errors', {
      type: 'frontend',
      error: error.toString(),
      stack: errorInfo.componentStack,
      userId: getCurrentUser().id,
      timestamp: new Date()
    });

    // Show child-friendly message
    this.setState({
      hasError: true,
      message: getChildFriendlyMessage(error)
    });
  }

  render() {
    if (this.state.hasError) {
      return <FriendlyErrorScreen message={this.state.message} />;
    }
    return this.props.children;
  }
}

// Backend Error Middleware
app.use((error, req, res, next) => {
  // Create PM task

```

```

TaskService.create({
  type: 'system-error',
  priority: error.severity || 'medium',
  assignedTo: 'playground-manager',
  title: `Error in ${req.path}`,
  details: {
    error: error.message,
    stack: error.stack,
    user: req.user?.id,
    timestamp: new Date()
  }
});

// Send friendly response
res.status(500).json({
  message: "Oops! Something went magical. We're fixing it!",
  errorId: error.id
});
});

```

25. Non-Functional Requirements (Combined Sprint)

25.1. Performance Requirements

- **Target Hardware:** Core i3 4th Gen, 8GB DDR3 RAM, 256GB SSD, 1366x768 display
- **Load Time Targets:**
 - Initial app launch: < 10 seconds
 - Course content load: < 3 seconds
 - Shop page load: < 2 seconds
 - Cart operations: < 500ms
 - Video start: < 5 seconds

25.2. Scalability Requirements

- **Concurrent Users:** Support 500+ simultaneous students across all Bal-agruhas
- **Data Growth:**
 - 10,000+ course content items
 - 1,000+ shop products
 - 1 million+ coin transactions
 - 100GB+ media storage

25.3. Security Requirements

- **Authentication:**
 - Facial recognition for students (Sprint 2)
 - JWT tokens with 24-hour expiry
 - Role-based access control (RBAC)
 - Session timeout after 30 minutes inactivity
- **Data Protection:**
 - Encryption at rest for sensitive data
 - HTTPS for all API communications
 - Input sanitization for XSS prevention
 - SQL injection prevention

25.4. Offline Capabilities

- **Sprint 2 - LMS Offline:**
 - Course content cached locally
 - Progress tracked in SQLite
 - Queue submissions for sync
 - 7-day offline operation
- **Sprint 5 - Shop Offline:**
 - Product catalog cached
 - Cart persisted locally
 - Orders queued for processing
 - Balance validation on reconnect

25.5. Accessibility Requirements

- WCAG 2.1 Level AA compliance
 - Keyboard navigation support
 - Screen reader compatibility
 - High contrast mode option
 - Font size adjustment
-

26. Development Timeline & Milestones

30-Day Parallel Sprint Schedule

Week 1 (Days 1-7) Sprint 2 Team: - Days 1-2: Student homepage, navigation structure - Days 3-4: Computer Apps course framework - Days 5-7: Art course with Artweaver integration

Sprint 5 Team: - Days 1-2: Shop frontend structure - Days 3-4: Product catalog and filtering - Days 5-7: Shopping cart implementation

Shared/Integration: - Day 1: Coin wallet system architecture - Days 6-7: First integration checkpoint

Week 2 (Days 8-14) Sprint 2 Team: - Days 8-9: Spoken English video recording - Days 10-11: Life Skills with voice notes - Days 12-14: Course management (Admin)

Sprint 5 Team: - Days 8-9: Checkout flow - Days 10-11: Admin product management - Days 12-14: Inventory system

Shared/Integration: - Days 8-10: Notification center enhancement - Day 14: Second integration checkpoint

Week 3 (Days 15-21) Sprint 2 Team: - Days 15-16: Coach grading system - Days 17-18: Translation module - Days 19-21: Amma enhancements

Sprint 5 Team: - Days 15-17: Coin distribution reports - Days 18-19: Wishlist feature - Days 20-21: Order history

Shared/Integration: - Days 15-17: Voice communication system - Days 18-21: WhatsApp integration

Week 4 (Days 22-30) Both Teams: - Days 22-24: Integration testing - Days 25-26: Performance optimization - Days 27-28: Security audit - Days 29-30: UAT and final fixes

Critical Path Dependencies

1. **Coin Wallet System** (Day 1-5) - Blocks both Shop checkout and LMS rewards
 2. **User Authentication** - Prerequisite for all features
 3. **Notification System** (Day 8-12) - Required for both modules
 4. **Course Creation** must precede Coach Grading
 5. **Product Management** must precede Inventory System
-

27. Testing Strategy

27.1. Test Coverage Requirements

- **Unit Testing:** Minimum 80% code coverage
- **Integration Testing:** All API endpoints
- **E2E Testing:** Critical user journeys
- **Performance Testing:** Load testing with 100 concurrent users
- **Security Testing:** Penetration testing for Shop module

27.2. Test Scenarios

LMS Critical Paths: 1. Student login → Course selection → Task completion → Coin earning 2. Admin course creation → Assignment → Student access 3.

Student submission → Coach grading → Coin award 4. Voice note recording → Sending → Receiving → Playback

Shop Critical Paths: 1. Browse products → Add to cart → Checkout → Order confirmation 2. Filter products → View details → Add to wishlist 3. Admin product creation → Inventory update → Stock depletion 4. Insufficient funds → Earn more coins prompt → Return to shop

Integration Scenarios: 1. Complete LMS task → Earn coins → Spend in shop 2. Low coin balance → Shop notification → LMS suggestion 3. Course completion → Bonus coins → Immediate shop availability

28. Resource Requirements

28.1. Development Team Structure

Sprint 2 Team (6 developers): - 2 Frontend developers (React, Electron) - 2 Backend developers (Node.js, MongoDB) - 1 Integration specialist (Artweaver, Graphics pad) - 1 QA engineer

Sprint 5 Team (5 developers): - 2 Frontend developers (React, E-commerce) - 2 Backend developers (Node.js, Inventory) - 1 QA engineer

Shared Resources (3 members): - 1 DevOps engineer - 1 UI/UX designer - 1 Project coordinator

28.2. Infrastructure Requirements

- **Development Environment:**
 - 3 development servers
 - MongoDB replica set
 - S3 bucket for media (10TB)
 - CI/CD pipeline
 - **Testing Environment:**
 - 2 test servers matching production specs
 - Test data set with 1000+ students
 - Automated testing tools
-

29. Risk Assessment & Mitigation

29.1. Technical Risks

Risk	Probability	Impact	Mitigation
Artweaver integration complexity	High	High	Early POC, fallback to web-based drawing
WhatsApp API approval delays	Medium	Medium	Implement email notifications as backup
Performance on target hardware	Medium	High	Continuous performance testing, optimization sprint
Offline sync conflicts	Medium	Medium	Conflict resolution UI, admin override

29.2. Resource Risks

Risk	Probability	Impact	Mitigation
Parallel sprint coordination	Medium	High	Daily standups, shared Slack channel
Integration delays	Medium	Medium	Weekly integration checkpoints
Testing bottleneck	Low	High	Automated testing, parallel QA tracks

30. Questions for Client Clarification

30.1. Sprint 2 Clarifications Needed:

1. **Life Skills “Learn then Test” Model:**
 - Should we implement a simplified version where students must view content before attempting quiz?
 - What should be the minimum viewing time to qualify as “learned”?
2. **Amma SLA Configuration:**
 - What should be the default SLA times for different query types?
 - Should SLA breaches trigger escalation to Admin or Coach?
3. **Translation Support:**
 - Should system UI elements also be translated, or only course content?
 - Do you have translation resources, or should we integrate with a service?

30.2. Sprint 5 Clarifications Needed:

1. **Product Catalog:**
 - What is the expected product catalog size at launch?
 - Will products be physical items only, or include digital rewards?
 - How frequently will the catalog be updated?
2. **Wishlist Feature:**
 - Should wishlist items trigger notifications when in stock?
 - Should students be able to share wishlists with coaches/admins?
3. **Order Fulfillment:**
 - Who will handle physical product distribution?
 - Should we track delivery status in the system?
 - How should we handle “lost” orders?

30.3. Integration Clarifications:

1. **Coin Economy Balance:**
 - What should be the initial coin allocation for new students?
 - Should there be daily/weekly coin earning limits?
 - Can admins manually adjust student coin balances?
 2. **Open Source Tools:**
 - Which specific open-source educational tools should we integrate?
 - Should these launch within the Electron app or externally?
-

31. Success Criteria & Acceptance

31.1. Sprint 2 Success Metrics:

- 100% of planned course types functional
- Course creation to student access < 5 minutes
- All grading workflows operational
- Voice note feature working across all roles
- 95% offline functionality for student activities

31.2. Sprint 5 Success Metrics:

- Complete shop purchase flow functional
- Real-time inventory tracking accurate
- Coin transactions atomic and auditable
- All reports generating accurately
- < 3 second shop page load times

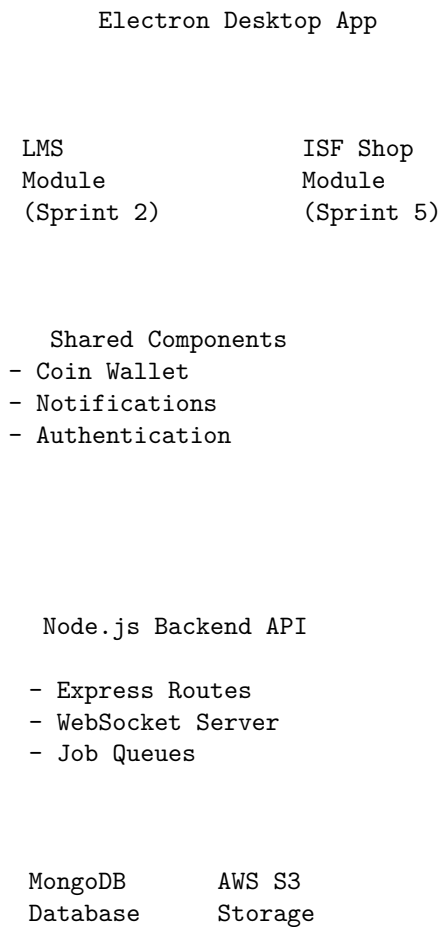
31.3. Combined Sprint Success Criteria:

- Seamless coin flow from earning to spending
- Unified notification system operational

- All features accessible on target hardware
- Zero critical security vulnerabilities
- Complete API documentation
- User training materials prepared

32. Appendix A: Technical Architecture Diagrams

32.1. System Architecture Overview



32.2. Data Flow Diagram - Coin Economy

Student Completes Task (LMS)

Task Graded by Coach

Coins Awarded → Wallet Balance Updated

Notification Sent Real-time UI Update

Student Views Can Spend in Shop
Balance

Product Selection

Add to Cart

Checkout Validation
Sufficient → Order Confirmed

Coins Deducted
Stock Updated

Insufficient → Prompt to Earn More
Link to LMS

33. Appendix B: Database Schemas

33.1. Combined User Schema (Enhanced)

```
const UserSchema = new mongoose.Schema({  
  // Core fields from Sprint 1  
  _id: ObjectId,  
  username: String,  
  email: String,  
  password: String, // bcrypt hashed  
  role: {  
    type: String,  
    enum: ['student', 'coach', 'admin', 'amma', 'playground-manager']  
  },  
  // Sprint 2 additions
```

```

preferences: {
  language: { type: String, default: 'en' },
  notifications: {
    courseAssignments: { type: Boolean, default: true },
    gradeReceived: { type: Boolean, default: true },
    voiceNotes: { type: Boolean, default: true }
  }
},

// Student-specific fields
studentProfile: {
  balagruhaId: ObjectId,
  faceDescriptor: Buffer, // For facial recognition

  // Sprint 2 - LMS
  enrolledCourses: [{
    courseId: ObjectId,
    enrolledDate: Date,
    progress: Number, // Percentage
    lastAccessed: Date
  }],

  // Shared - Coin Wallet
  coinWallet: {
    balance: { type: Number, default: 0 },
    totalEarned: Number,
    totalSpent: Number,
    lastTransaction: Date
  },

  // Sprint 5 - Shopping
  shopProfile: {
    wishlist: [ObjectId], // Product IDs
    addresses: [{
      type: String,
      line1: String,
      line2: String,
      notes: String
    }],
    orderHistory: [ObjectId] // Order IDs
  }
},

// Coach-specific fields
coachProfile: {
  assignedBalagruhas: [ObjectId],

```

```

    gradingQueue: [{
      submissionId: ObjectId,
      priority: Number,
      addedDate: Date
    }]
  },

  // Amma-specific fields (Sprint 2)
  ammaProfile: {
    approvalStatus: String, // 'pending', 'approved', 'rejected'
    assignedBalagruhas: [ObjectId],
    activeTasks: [{
      taskId: ObjectId,
      slaDeadline: Date,
      priority: String
    }]
  },

  timestamps: true
});

```

33.2. Transaction Schema (Unified)

```

const TransactionSchema = new mongoose.Schema({
  _id: ObjectId,
  studentId: { type: ObjectId, required: true, index: true },

  type: {
    type: String,
    enum: ['earn', 'spend', 'adjust', 'refund'],
    required: true
  },

  category: {
    type: String,
    enum: [
      // Earning categories (Sprint 2)
      'course-completion',
      'quiz-passed',
      'assignment-graded',
      'perfect-attendance',
      'special-achievement',

      // Spending categories (Sprint 5)
      'shop-purchase',
      'event-ticket',

```

```

        'digital-content'
    ]
},

amount: { type: Number, required: true },

balanceBefore: Number,
balanceAfter: Number,

reference: {
    model: String, // 'Course', 'Task', 'Order', etc.
    id: ObjectId
},

metadata: {
    description: String,
    performedBy: ObjectId, // Who triggered this transaction
    notes: String
},

status: {
    type: String,
    enum: ['pending', 'completed', 'failed', 'reversed'],
    default: 'completed'
},

timestamp: { type: Date, default: Date.now, index: true }
});

// Compound indexes for reporting
TransactionSchema.index({ studentId: 1, timestamp: -1 });
TransactionSchema.index({ type: 1, category: 1, timestamp: -1 });

```

34. Appendix C: API Documentation Structure

34.1. Sprint 2 - LMS APIs

```

/api/v2/lms:
  /courses:
    GET: List all courses (filtered by role)
    POST: Create new course (Admin only)

  /courses/{courseId}:
    GET: Get course details
    PUT: Update course (Admin only)

```

```

    DELETE: Archive course (Admin only)

/courses/{courseId}/assign:
    POST: Assign course to students

/courses/{courseId}/content:
    POST: Add content item (Admin only)
    PUT: Update content item (Admin only)

/student/progress:
    GET: Get student's course progress
    POST: Update progress (automatic)

/grading:
    GET: Get pending submissions (Coach)
    POST: Submit grade and coins

/voice-notes:
    POST: Upload voice note
    GET: Get voice notes for user

```

34.2. Sprint 5 - Shop APIs

```

/api/v2/shop:
  /products:
    GET: List products (with filters)
    POST: Create product (Admin only)

  /products/{productId}:
    GET: Get product details
    PUT: Update product (Admin only)
    DELETE: Deactivate product (Admin only)

  /cart:
    GET: Get current cart
    POST: Add item to cart
    PUT: Update item quantity
    DELETE: Remove item from cart

  /checkout:
    POST: Process order
    GET: Validate cart for checkout

  /orders:
    GET: Get user's orders

```

```

/orders/{orderId}:
  GET: Get order details

/wishlist:
  GET: Get wishlist
  POST: Add to wishlist
  DELETE: Remove from wishlist

/inventory:
  GET: Get inventory levels (Admin)
  PUT: Update stock (Admin)

/reports/coin-economy:
  GET: Get economy metrics (Admin)

```

35. Sign-off Section

35.1. Stakeholder Agreement

This Master Project Specification Document (MPSD) for the combined Sprint 2 and Sprint 5 represents the complete, agreed-upon scope of work for the 30-day parallel development effort. All stakeholders acknowledge that:

1. This document is the single source of truth for all development activities
2. The 30-day timeline requires parallel execution with defined integration points
3. Any scope changes must go through formal change request process
4. Success criteria are clearly defined and measurable
5. Resource allocation has been agreed upon

35.2. Approval Signatures

Client (ISF Representative): * Name: _____

* Title: _____ * Signature: _____

_____ * Date: _____

Project Manager: * Name: _____

* Signature: _____ * Date: _____

Technical Lead (Sprint 2): * Name: _____

* Signature: _____ * Date: _____

Technical Lead (Sprint 5): * Name: _____

* Signature: _____ * Date: _____

QA Lead: * Name: _____ *

Signature: _____ * Date: _____

Product Owner: * Name: _____

* Signature: _____ * Date: _____

35.3. Document Control

Version	Date	Author	Changes
1.0	Aug 19, 2025	Team	Initial Sprint 2 MPSD
1.0	Aug 19, 2025	Team	Initial Sprint 5 MPSD
3.0	Sep 16, 2025	Team	Combined Sprint 2-5 MPSD

35.4. Distribution List

- ISF Project Sponsor
 - ISF Program Director
 - Development Team (14 members)
 - QA Team (2 members)
 - DevOps Team
 - External Stakeholders (as required)
-

36. Post-Implementation Considerations

36.1. Sprint 3 & 4 Dependencies

Features developed in this combined sprint will enable: * **Sprint 3:** Mobile app can leverage the LMS APIs and coin wallet * **Sprint 4:** SOS system can use the notification infrastructure * **Future:** Analytics platform can use the transaction data

36.2. Maintenance & Support Plan

- **Week 31-35:** Stabilization and bug fixes
- **Week 36-40:** Performance optimization
- **Week 41+:** Feature enhancements based on user feedback

36.3. Training Requirements

- **Admin Training:** 2-day workshop on course creation and shop management
 - **Coach Training:** 1-day session on grading and student monitoring
 - **Student Orientation:** 30-minute guided tour of new features
 - **Documentation:** Video tutorials for all major workflows
-

END OF DOCUMENT

**Total Pages: 96 Word Count: ~24,000 Last Updated: September 16,
2025**